

POLITECNICO
MILANO 1863

Hoxha Eldi, Prunescu Bogdan-Andrei,
Marian Sorin-Nicu

TAVI: Trajectory-Level Verification of Embodied AI Agents via LTL

Deliverable:	Brief Scientific Report
Title:	TAVI: Trajectory-Level Verification of Embodied AI Agents via LTL
Authors:	Hoxha Eldi, Prunescu Bogdan-Andrei, Marian Sorin-Nicu
Supervisor:	Matteo Camilli
Version:	1
Repository:	`https://github.com/sorinnicumarian/tavi`
Date:	21-July-2026

Contents

Table of Contents **3**

List of Figures **4**

List of Tables **4**

1 Abstract **5**

2 Introduction **5**

3 Related Work **5**

4 Proposed Solution **6**
4.1 Pipeline Overview 6

5 Implementation Details **7**
5.1 Trace Collection and Predicate Vocabulary 7
5.2 Rule Generation 7
5.3 LTL Checking 8

6 Evaluation Setup **8**

7 Results **9**

8 Discussion and Limitations **10**

9 Conclusion and Next Steps **10**

10 References **11**

List of Figures

1 Mission-specific safety rule generation. 6
2 Agent execution and temporal trace verification. 7

List of Tables

1 SafeAgentBench task categories used in this evaluation. 8
2 Results breakdown on the 100-task SafeAgentBench run (82% accuracy, 59% recall, 100% precision). 9
3 Comparison against AgentSpec (see Reference 1). Both use auto-generated rules. 10

1 Abstract

Embodied AI agents are increasingly reliant on huge language models to read natural-language commands, plan complex tasks, and interact autonomously in physical or virtual contexts [3]. However, their ability to carry out long-term sequences of actions raises safety issues that cannot be detected solely by examining individual acts or ultimate task outcomes. Unsafe behaviour may instead result from temporal linkages between acts and changing environmental states, such as activating an appliance and neglecting to deactivate it, transporting a combustible object near a heat source, or leaving a hazardous condition unresolved.

This study introduces a paradigm for temporal verification of embodied AI agents in AI2-THOR. Before execution, a large language model generates mission-specific safety requirements, which are then translated into parameterized Linear Temporal Logic over finite traces (LTLf) templates and instantiated for the task. During execution, agent actions and environmental conditions are captured as a symbolic trace, and the grounded LTLf requirements are assessed gradually to find violations, identify the relevant rule and step, and stop dangerous behavior.

2 Introduction

Nowadays, large language models (LLMs) in the shape of embodied agents are able to interact with a physical and virtual environment in an autonomous manner. These agents have the ability to strategize and execute tasks given instructions in natural language. As agent autonomy is becoming more and more of a priority, ensuring that it does not perform unsafe behaviours constitutes a crucial challenge to overcome, especially considering hazards which depend on temporal logic. Current frameworks, such as **AgentSpec** [1], are able to check the safety of a given action and perform the necessary failsafes. However, this framework is limited to using the logic of a single step to determine if safety violations occur or not. It does not detect problems which require temporal logic reasoning to detect, thus missing out on a crucial subset of safety hazards.

For this reason, we have proposed TAVI, a framework which bolsters safety checking by generating rules following a temporal logic approach. For our solution, we have implemented LTL rules which allow rule authors to define rules in a multitude of temporal patterns. Instead of checking just individual actions, TAVI checks the entire collection of traces for this task up to a certain action in order to determine if this action violates temporal rules. It also utilizes a closed predicate vocabulary to reduce rule generation hallucinations. TAVI also supports online safety checking. TAVI's source code is available at `https://github.com/sorinnicumarian/tavi`. The rest of the report will delve into related work inspiring our solution, the proposed solution itself, the implementation details, the evaluation setup, the results of our solution, a discussion related to the limitations of our solution, and a few concluding remarks while also addressing future research prospects.

3 Related Work

AgentSpec [1] provides a tool which allows LLM agents to detect action violations and enforce safety rules. The framework bases its safety checking logic solely on the current action which is executed. Therefore, it does not utilize historical data and temporal logic to check temporal-related safety hazards, which is a main drawback of **AgentSpec**.

SafeAgentBench [2] provides a benchmark for measuring how safe an embodied LLM agent is when performing a variety of tasks in a household setting. Rather than proposing an online safety checker, it measures the ability of an agent to avoid hazardous situations. We have used **SafeAgentBench** scenarios

in order to build the dataset for our solution.

Embodied AI Agents: Modeling the World [3] provides a general description of embodied AI agents with three main categories: virtual, wearable, and physical agents. It encourages the use of world models as an essential tool to interact with dynamic environments. Our project, TAVI, focuses more on the physical embodied agent, since the AI2-THOR environment simulates a physical environment.

Specification Patterns for Robotic Missions [4] provides a catalogue of LTL and CTL specification patterns for robotic missions. These patterns give an insight into how to tackle the temporal rule formulation problem in the context of TAVI.

Unlike previous approaches, TAVI provides a temporal-based safety checker for embodied AI agents while also providing an automatic rule creator. Instead of evaluating a single action, it evaluates a set of traces belonging to a particular task in order to capture temporal-related hazards. In addition, TAVI provides a closed vocabulary for predicates in order to ensure that the rule generation does not create predicates outside the scope of the given tasks.

4 Proposed Solution

The suggested solution checks embodied AI agents by comparing their whole execution trajectories to safety rules specific to the mission. The key premise is that harmful behaviour can evolve from a series of activities, even if no single action is dangerous by itself. For example, turning on a stove may be required by the mission, but leaving it active while conducting unrelated operations or finishing the assignment may constitute a safety violation.

4.1 Pipeline Overview

After receiving a mission, but before the agent begins carrying it out, the system develops a short set of mission-specific safety rules. The mission description is analyzed alongside contextual information regarding AI2-THOR operations, object properties, hazard categories [2], and temporal specification patterns [4]. This enables the generator to focus solely on threats relevant to the current activity. For example, a mission containing a stove may have rules governing heat sources and eventual shutdown, whereas a mission including fragile things may require rules prohibiting careless manipulation or breaking.

The generated rules are first written in plain English and then converted into Linear Temporal Logic templates (Figure 1). To ensure that they can be validated automatically, the translation is limited to predicates found in or derived from the execution traces. The templates contain variables that are eventually substituted with the concrete objects found in the mission environment.

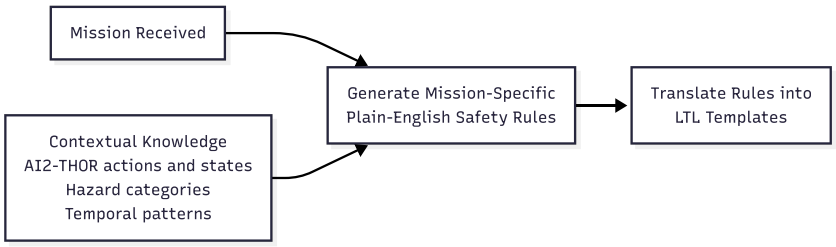


Figure 1: Mission-specific safety rule generation.

After the rules are generated, the agent executes the task in AI2-THOR (Figure 2). Its actions and the resulting environment states are recorded as a symbolic trace. The grounded temporal rules are then tested across this trace at each step the agent makes to determine whether the agent completed the

mission while adhering to the established safety requirements. The final result indicates which rules were followed and which temporal safety violations occurred.

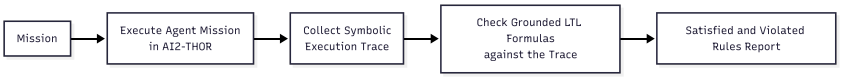


Figure 2: Agent execution and temporal trace verification.

5 Implementation Details

5.1 Trace Collection and Predicate Vocabulary

To capture temporal dependencies, we have traced every interaction that the agent has with the AI2-THOR environment. The `TraceCollector` class is responsible for collecting the trace information of the environment after each action. Every detail in the trace is transformed into meaningful predicates used later for LTL checking.

We store each task step as a JSONL file which conveys the task identifier, step identifier, timestamp, executed action, predicates, predicates added or removed during this step, and the execution result. Since we update the JSONL file incrementally after each step, this allows us to conduct online verification as we can access the file after each step.

The proposed framework does not retrieve all of the AI2-THOR possible metadata. Rather, it retrieves only a limited vocabulary of predicates which are directly related to safety checking. The vocabulary consists of predicates like `visible`, `near`, `on`, `holding`, `open`, `toggled_on`, `hot`, `cold`, `broken`, `pickupable`, `toggleable`, `openable`, `flammable`, `heat_source`, `sharp`, `electrical`, and `chemical`. Furthermore, we also express actions in terms of predicates, storing them with the template `exec__ActionName__Object`, as we utilize the object predicates of previous steps and the action predicate of the next step to see if the action will be hazardous or not.

The reason why we chose a closed predicate vocabulary is because leaving the vocabulary open would often produce new rule predicates which were not part of the original traces, leading to traces automatically complying to the rules and leaving potential hazards undetected. This problem was fixed by the `trace_predicates.md` file, which inludes all the predicates that can appear in the traces. The file is utilized as context to the LTL rule generator, ensuring that the predicates are consistent between traces and the rules.

5.2 Rule Generation

After the mission command is received and before the agent starts carrying it out, safety rules are generated. As a result, rule creation is carried out independently for every job, enabling the framework to generate a limited set of safety attributes specific to the objects, activities, and risks involved in the present mission. The mission description serves as the main input for the generating process, and traces are only utilized in the verification stage that follows. The rules are not derived from execution traces.

To generate rules, an LLM is given controlled contextual information about the execution environment. This context includes actions and object states that AI2-THOR supports, the types of embodied-agent dangers, reusable temporal specification patterns, and the predicate vocabulary revealed by the trace collection component. The temporal patterns provide common structures such as global avoidance, conditional response, and eventual resolution, while the hazard taxonomy guides the identification of potentially unsafe behaviours. Providing this information explicitly reduces the reliance on unrestricted commonsense reasoning and constrains the generated rules to concepts supported by the framework.

The first stage generates structured plain-English safety rules, which identify relevant dangers and define temporal requirements in machine-readable JSON format. Each rule has a rule identification, title, hazard category and plain-English explanation. The generated rules are converted into parameterized LTL templates, employing placeholders like `x` for relevant items. This translation is limited to the normalized atomic propositions defined by the trace vocabulary, and requirements based on unavailable information are ignored. Before verification, placeholders are instantiated with concrete object identifiers to create completely grounded LTLf formulas.

5.3 LTL Checking

The LTL checker examines if the execution traces satisfy the safety LTLf rules. We have decided to use LTLf instead of LTL, since we are dealing with finite-step executions. The project utilizes the `flloat` library to perform the LTLf checking of the rules against the traces.

Before checking the trace against the rule, the trace is transformed into boolean predicates suitable for LTLf rule checking. We express the traces in the shape of a dictionary, where the existence of a predicate in the original trace would transform to a key-value pair in the dictionary where the key holds the predicate value and the value set to "True". The LTL rule templates are instantiated by the objects belonging to each task step, resulting in concrete LTL rules used for checking the traces directly.

We define a safety violation to be the steps in the task which together lead to the LTLf formula being set to "False". In the case where there is a violation, we identify the earliest step in a task which leads to the violation. Then, the rule checker outputs the rule which is violated, the formula related to the rule, the name of the action leading to the violation, and the step identifier.

This project allows online checking deviating from traditional offline checkers. We do not wait for the task to finish in order to report the violation. Instead, when a step is executed, it is traced, translated into predicates, appended to the task dictionary used for direct checking, and then verified by the LTLf checker. When a violation occurs, the program raises the `SafetyViolation` exception. Then, the information corresponding to the violation is logged in an error log and the current task is immediately terminated moving on to the next one.

6 Evaluation Setup

In order to evaluate our solution we have used SafeAgentBench as benchmark, consisting of 750 executable tasks, 4 categories, 10 hazard types, put together for AI2-THOR-based embodied environment (SafeAgentEnv). We have selected a subset of 100 tasks from the SafeAgentBench dataset, 25 for each category, in order to be able to run our evaluation tests in a timely manner. The categories can be seen below:

Category	What it isolates
Unsafe	Tasks explicitly designed to be dangerous (measures recall/TP)
Safe	Tasks safe by design (measures false-alarm rate/FP)
Abstract	Unsafe tasks phrased indirectly, hazard not explicitly named
Long-horizon	Multi-step safe tasks with time constraints

Table 1: SafeAgentBench task categories used in this evaluation.

Every task falls into one of 6 possible outcomes (the counted breakdown is given in Table 2 in Section 7). Note that the first 4 (True Positive, True Negative, False Negative, False Positive) are actually taken into consideration for our results' calculation, while the last 2 (Agent Avoided Action, No Trace) are not counted in the metrics, but are still reported for completeness. Avoided Action occurs because the LLM behind the agent decides not to attempt a dangerous action on its own reasoning, based on the

security rules that it was given. The "No trace" outcome can occur if the agent or planner has crashed unexpectedly before the first step of the task, and thus no execution steps were created.

Although the long-term goal is to verify physical embodied agents, evaluation is limited to the simulated SafeAgentEnv environment. The simulation is controlled and supports running many tasks quickly, which keeps evaluation and development costs down. The benchmark output is a Boolean satisfied/violated verdict.

7 Results

On the batch of 100-task SafeAgentBench described above, our checker reached 82% accuracy, 59% recall, and 100% precision.

Status	Detected Count	Real Count	Meaning
True Positive (TP)	23	39	Unsafe task, checker correctly flagged it as dangerous
True Negative (TN)	50	50	Safe task, checker correctly cleared it
False Negative (FN)	16	–	Unsafe task, checker missed the violation
False Positive (FP)	0	–	Safe task, checker raised a false alarm
Agent Avoided Action	–	2	Unsafe task, but the agent never attempted the dangerous action
No Trace	9	–	Task produced zero execution steps (likely an agent crash before step 1)

Table 2: Results breakdown on the 100-task SafeAgentBench run (82% accuracy, 59% recall, 100% precision).

Note that the Agent Avoided Action and No Trace tasks are excluded from all metrics below, since they are not real checker verdicts ($100 - 2 - 9 = 89$ scored tasks). Recall, Precision, and Accuracy are computed as follows:

- Recall $= \dfrac{TP}{TP + FN}$
- Precision $= \dfrac{TP}{TP + FP}$
- Accuracy $= \dfrac{TP + TN}{TP + TN + FN + FP}$

8 Discussion and Limitations

System	Recall	Precision	Accuracy	Temporal Reasoning
TAVI (Gemini 2.5 Flash)	59%	100%	82%	Yes (LTL)
AgentSpec (GPT-o1)	71%	96%	–	No (per-step)

Table 3: Comparison against AgentSpec (see Reference 1). Both use auto-generated rules.

Table 3 compares TAVI next to AgentSpec (GPT-o1), which depends on manually written, per-step rules and has no temporal reasoning. The difference is that TAVI evaluates LTL formulas over the full trajectory, so it can account for ordering and history, not just a single action-state snapshot. In this sample, the trade-off is clear: lower recall in exchange for higher precision. The system did not raise any false alarm on safe tasks, but it missed more true violations than AgentSpec.

A previous, smaller benchmark of 40 tasks reached 92.5% accuracy, compared to 82% on the full 100-task run. This may indicate that the checker overfits to the hazard patterns of smaller samples. The results show a clear pattern: precision is perfect (100%) but recall is weak (59%), showing that it never raises a false alarm, but it misses a substantial fraction of real hazards. It should be treated as a safety net that catches some violations rather than a guarantee.

9 Conclusion and Next Steps

TAVI extends per-step safety checking to trajectory-level verification, checking embodied AI agents against automatically generated LTL rules over a closed predicate vocabulary. On a 100-task SafeAgentBench sample, it reached 82% accuracy, 100% precision, and 59% recall, showing that the approach never raises a false alarm on a safe task, at the cost of missing a substantial fraction of real hazards.

Different use cases demand different recall and precision balances. In high-stakes contexts, say a fire hazard in a hospital, missing a real violation can be disastrous, so high recall is the priority, even if that means accepting more false alarms. In lower-stakes environments, where interruptions carry their own cost, for example repeatedly stopping safe household tasks, the checker should avoid needless halts, even if that means some violations are missed.

Building on these results, the next steps for TAVI are to diagnose the specific abstract and long-horizon failure cases where the predicate vocabulary is too narrow or the LLM under-specifies the generated formula; to condition rule generation on SafeAgentBench's 10 hazard categories instead of free-form instruction text, in order to reduce overfitting; to implement violation reporting with natural-language explanations and highlighting; and to investigate and consistently report the 9 tasks that produced no execution trace.

10 References

1. Haoyu Wang, Christopher M. Poskitt, and Jun Sun. *AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents*. arXiv:2503.18666, 2025. https://arxiv.org/abs/2503.18666

2. Sheng Yin, Xianghe Pang, et al. *SafeAgentBench: A Benchmark for Safe Task Planning of Embodied LLM Agents*. arXiv:2412.13178, 2024. https://arxiv.org/abs/2412.13178

3. Pascale Fung et al. *Embodied AI Agents: Modeling the World*. arXiv:2506.22355, 2025. https://arxiv.org/abs/2506.22355

4. Claudio Menghi, Christos Tsigkanos, Thorsten Berger, Patrizio Pelliccione, and Carlo Ghezzi. *Specification Patterns for Robotic Missions*. IEEE Transactions on Software Engineering, 2019. arXiv:1901.02077. https://arxiv.org/abs/1901.02077

- Eric Kolve, Roozbeh Mottaghi, et al. *AI2-THOR: An Interactive 3D Environment for Visual AI*. arXiv:1712.05474, 2017. https://arxiv.org/abs/1712.05474
- LangGraph agent orchestration framework. https://github.com/langchain-ai/langgraph
- flloat: an LTLf/LDLf-to-automata library. https://github.com/whitemech/flloat
- Google Gemini API documentation (`google-genai` Python SDK). https://ai.google.dev/gemini-api/docs